

INTRODUCING DR.C: A DOMAIN-SPECIALIZED MULTI-AGENT AI FOR DESIGNING CSOUND INSTRUMENTS AND PLUGINS AND DEVELOPING EDUCATIONAL, GENERATIVE AND AUDIO REACTIVE WEB APPLICATIONS

Richard Boulanger

Berklee College of Music
Boston, MA, USA
rboulanger@berklee.edu

Mateo Larrea

CCRMA, Stanford University
Stanford, CA, USA
mlarrea@stanford.edu

ABSTRACT

We present Dr.C, an open-source AI system specialized for Csound that addresses fundamental limitations in general-purpose AI coding assistants. Dr.C implements a domain-specialized architecture combining multi-agent orchestration, curated knowledge retrieval, semantic validation, and an alternatives-first design paradigm that positions the AI as collaborator rather than autonomous generator. Key contributions include Csound-aware structural patching, contract validation beyond syntax checking, multi-tier knowledge retrieval, and design tree exploration. Dr.C is freely available under MIT license as a template for building domain-specialized AI assistants for FLOSS audio development.

Keywords: Csound, Domain-Specialized, Multi-Agent, Dr.C, Educational-Scaffolding

1. INTRODUCTION

Large language models (LLMs) like GPT-4 and Claude have enabled widespread adoption of AI-assisted programming. However, applying these general-purpose systems to computer music reveals limitations: syntax hallucination (Csound 6 vs. 7 incompatibilities), lack of curated domain knowledge, premature optimization, missing semantic validation, and poor educational scaffolding [1]. For Csound—a mature FLOSS audio language with 35+ years of evolution—these issues are acute, as LLMs trained on web-scraped code cannot navigate its significant version changes.

1.1. Contributions

We present **Dr.C**, a domain-specialized AI system for Csound development with the following contributions:

1. **Alternatives-first design paradigm:** A novel interaction model where the AI proposes 2–4 design alternatives varying by technique, character, and complexity before executing, keeping humans in creative control
2. **Multi-tier knowledge retrieval:** Combines opcode quick-reference cards, working example CSDs, prose from synthesis textbooks, and knowledge graph traversal with domain-aware query routing
3. **Semantic validation framework:** Beyond syntax checking—validates Cabbage widget bindings, parameter smoothing, UI layout, and user-defined locked parameters
4. **Multi-agent orchestration:** Specialized sub-agents for synthesis, effects, and modulation that execute concurrently on independent sub-tasks
5. **Session-based workspace isolation:** Safe experimentation with sandboxed temporary copies and explicit save/discard workflow
6. **Design tree exploration:** Non-linear version control capturing all explored alternatives with sonic characterization metadata

Dr.C is intentionally scoped to Csound (.csd file) synthesis, Cabbage plugin development, and educational exploration—enabling deep specialization that outperforms general-purpose tools.

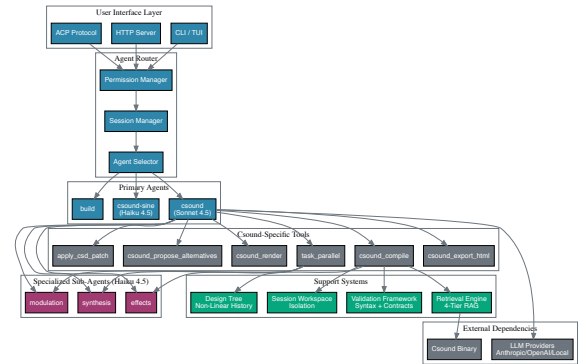


Figure 1: *Dr.C* system architecture showing user interface layer, agent router, primary agents, specialized sub-agents, Csound-specific tools, and support systems including 4-tier RAG, validation framework, session workspace, and design tree.

2. SYSTEM ARCHITECTURE

2.1. Multi-Agent System

Dr.C implements a hierarchical multi-agent architecture with role-specific agents (Figure 1):

Primary Agents:

- **csound** (Claude Sonnet 4.5): Full-featured design agent with alternatives workflow and RAG
- **csound-sine** (Claude Haiku 4.5): Lightweight fast-edit agent for quick fixes
- **build**: General-purpose agent for non-Csound tasks

Specialized Sub-Agents (Claude Haiku 4.5 for cost/speed):

- **csound-synthesis**: Oscillators, FM/AM, additive/subtractive, physical modeling
- **csound-effects**: Reverb, delay, filters, distortion, EQ, dynamics
- **csound-modulation**: Envelopes, LFOs, control signals, MIDI/OSC

The permission system provides fine-grained control, denying all operations by default, allowing file reading and Csound tools, restricting edits to .csd files only, and prompting for access to external directories.

2.2. Domain-Specific Tools

Dr.C extends the base agent framework with Csound-specific tools:

csound_compile: Syntax validation with structured diagnostics including severity, line/column numbers, error messages, opcode context, error category, and actionable suggestions.



Figure 2: Comparison of traditional AI workflow (generate-and-refine, reactive) versus Dr.C's alternatives-first approach (proactive choice, design space exploration).

csound_render: Renders CSD to WAV with configurable duration (default 10s), optional automatic playback, WAV header parsing for metadata, and timeout protection.

csound_propose_alternatives: Presents 2–4 alternatives varying by technique, character, complexity. Each includes label, sonic description, approach, and ready-to-apply patch. Creates design tree branch nodes for version control.

apply_csd_patch: Structure-aware patching that validates section boundaries (CsInstruments, CsScore, CsOptions, Cabbage), enforces locked parameter constraints, and uses file locking for consistency.

csound_export_html: Exports to standalone HTML using @csound/browser CDN with self-contained play/stop controls, code display, and console output.

task_parallel: Launches 2–4 independent sub-agents in parallel, dramatically reducing latency (e.g., 10s wall-clock vs. 30s sequential for 3 tasks), aggregating results and reporting failures gracefully.

2.3. Knowledge Retrieval System

Dr.C implements a four-tier retrieval-augmented generation (RAG) system:

Tier 0: Opcode Quick-Reference Cards — Compact syntax cards for all Csound opcodes with parameters, ranges, common patterns, and pitfalls, domain-tagged for relevance scoring.

Tier 1: CSD Example Index — Indexed working .csd files from CsoundQt examples [24], Csound Manual examples [14], FLOSS Manual examples [16], Boulanger Csound Instrument Catalog, Cabbage examples [19], Iain McCurdy Instrument Collection [23], and Gogins collection [18]. Metadata searchable via Orama (BM25 + embeddings), full content lazy-loaded on demand, linked to knowledge graph.

Tier 2: Prose Knowledge — Chunked content from *The Csound Book* [1], *Csound: A Sound and Music Computing System* [2], Csound 7 reference manual [14], FLOSS Manual [15], Csound Journal (all issues) [21], Csound Magazine (all issues) [22], Cabbage Manual [19], and Cabbage Forum [20].

Tier 3: Knowledge Graph — Ontology linking opcodes to synthesis techniques, related opcodes, example CSDs, textbook sections, and acoustic theory.

Query routing uses domain-specific heuristics: technical syntax errors route to Tier 0 (opcode cards), aesthetic requests ("warm reverb") route to Tier 2 (prose), and troubleshooting routes to Tier 1 (working examples).

3. ALTERNATIVES-FIRST DESIGN PARADIGM

Traditional AI assistants generate a single solution, forcing users into a reactive refinement loop. Dr.C introduces an **alternatives-first** approach where the system proposes multiple design options *before* execution (Figure 2).

3.1. Design Principles

1. **Proactive Choice:** User selects from alternatives rather than reacting to AI output
2. **Design Space Exploration:** Alternatives vary along orthogonal dimensions (technique, timbre, complexity)

3. **Educational Scaffolding:** Each alternative includes technical explanation and parameter semantics
4. **Creative Agency:** Human maintains control over aesthetic decisions

3.2. Example Workflow

User request: "Add reverb to this synth"

Dr.C proposes three alternatives:

1. **Plate Reverb (Warm, Dense):** Uses `reverb~` with long decay, low-pass filtering. Creates smooth, warm spatial image.
2. **Schroeder Reverb (Bright, Metallic):** Nested allpass filters with short delays. Bright, diffuse character for percussive sources.
3. **Spring Reverb (Dark, Vintage):** Physical model with resonant modes. Colored, vintage character with boingy artifacts.

Each alternative includes implementation approach, sonic description, and unified diff patch. User selects, system applies and renders immediately.

4. SEMANTIC VALIDATION FRAMEWORK

Beyond syntax checking, Dr.C implements semantic validation for common error patterns:

4.1. Cabbage Contract Validation

Cabbage plugins require strict coupling between UI widgets and instrument code. Dr.C validates:

- Every `channel ("X")` widget has matching `chnget "X"` in orchestra
- All `chnget` calls use `port` smoothing to prevent zipper noise
- Widget ranges match instrument parameter expectations
- Layout geometry (no overlapping widgets, bounds checking)

4.2. Csound 7 Correctness Rules

Common AI mistakes from outdated training data:

- Variable prefix must match opcode output rate (`aOut = oscili(...)` not `kOut`)
- Always set `0dbfs = 1` in orchestra header
- UDO type strings required in old-style: `opcode Name, a, ak`
- Use `cpsmidinn(noteNum)` to convert MIDI notes, not `cpsmidi()`
- `expseg` values must all be > 0 (use 0.0001 instead of 0)
- Global audio accumulators must be cleared every k-cycle

Validation catches common error categories during development: Cabbage contract violations (mismatched channel declarations), silent output bugs (missing `out()` calls), Csound 6/7 syntax incompatibilities, and runtime errors (`expseg` zero values, uncleared accumulators). These checks prevented numerous bugs across the six deployed applications.

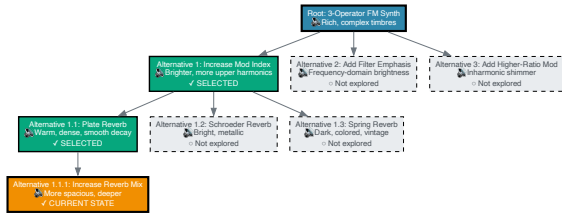


Figure 3: Design tree showing non-linear exploration of FM synthesis variations. Root node: 3-operator FM synth. Branch 1: modulation index variations. Branch 2: reverb type alternatives. Current state highlighted, unexplored alternatives shown as potential paths.

5. DESIGN TREE EXPLORATION

Dr.C captures exploration history as a non-linear design tree (Figure 3), enabling backtracking, A/B comparison, and documentation of creative process.

Each node stores:

- CSD snapshot hash (efficient deduplication)
- Sonic characterization metadata ("warm", "bright", "resonant")
- Alternatives presented (including not-selected branches)
- User selection reasoning
- Audio render hash (links to WAV file)

Design trees enable pedagogy: students submit trees showing exploration process, not just final code. Instructors assess design thinking, not just implementation.

6. EDUCATIONAL DEPLOYMENT

Dr.C is currently being used in Electronic Production and Design courses at Berklee College of Music—Composition (EP461), DSP (EP413), and Csound (EP337)—as well as in private instruction with high school, undergraduate, and graduate students by both authors. The scope, quality, and quantity of student work using Dr.C is significant, with overwhelmingly positive feedback. Students report that Dr.C’s ability to accelerate development enables them to explore more ambitious projects, which increases their motivation to learn synthesis concepts more deeply.

6.1. Vibe Coding Pedagogy

"Vibe coding" allows students to describe aesthetic goals ("warm, resonant filter") rather than technical implementation. Dr.C proposes alternatives with technical scaffolding including explanations, parameter meanings, and signal flow diagrams.

Block Diagrams: Dr.C generates ASCII signal flow diagrams demystifying code structure:

```
[Input] -> [FM Osc] -> [Filter] -> [Reverb] -> [Out]
      |           |           |           |
      |           |           |           |
[Mod Env] [Cutoff] [Decay Time]
```

From Conception to Production: Students co-design custom synthesizers and audio processors with Dr.C, then immediately deploy them as audio-reactive web applications or VST/AU plugins using Cabbage. This complete workflow—from aesthetic concept to working production tool—happens within hours. Students create music using instruments and effects they designed themselves, fostering unprecedented ownership, pride, and creative accomplishment. The ability to compose with personally crafted intelligent generative systems transforms the educational experience from passive learning to active tool-building, fundamentally changing how students engage with synthesis and driving creative work to new levels of ambition and sophistication.

6.2. Educational Outcomes

Dr.C maintains students as active participants in the collaborative design process—the user-designer remains in the loop at every decision point. Students immediately use their co-designed instruments to create music, play with parameters, explore sonic possibilities, and learn through direct experience. This instant creative feedback loop—design, test, compose—fundamentally transforms the learning experience.

Transparency and Learning: Dr.C teaches while building. All generated code appears directly in the application UI for immediate review, modification, and execution. The design tree preserves complete history, allowing students to retrace any previous step, compare alternatives, and understand evolution of their design. Dr.C generates detailed technical documentation and tutorials explaining parameter meanings, signal flow, and synthesis concepts, enabling students to study and internalize the reasoning behind each design decision.

The alternatives-first approach provides pedagogical scaffolding that helps students understand synthesis concepts while accelerating development workflows. Students can explain the code Dr.C generates and understand the technical reasoning behind design decisions. Because Dr.C generates working, documented, explorable systems instantly, students can focus on creative goals while simultaneously absorbing technical knowledge through guided exploration rather than syntax memorization.

6.3. Pedagogical Challenges and Curriculum Adaptation

However, this approach presents significant educational challenges that require careful curriculum design. Students could potentially "push the play button" and let Dr.C generate everything without understanding underlying algorithms, opcodes, or synthesis fundamentals—analogue to a musician who cannot play their instrument. Several critical questions emerge: How do we teach? What do we teach? How do we evaluate learning? How do we measure genuine understanding versus surface-level generation?

Foundational Skills First: Critically, Dr.C is introduced only in the final four weeks of the semester, after students have already established core competencies. Csound students (EP337) have designed basic instruments and effects manually and converted them to plugins via Cabbage. DSP students (EP413) have learned and composed with spectral processing tools [3, 4, 5] before building their own with Dr.C. Composition students (EP461) have completed ambient compositions [29, 11, 12], Fibonacci-based FM studies, musique concrète projects, and electronic music techniques [6, 7, 8, 9, 10], applying principles of algorithmic composition, sound design, and musical form before using AI assistance. This sequencing ensures Dr.C accelerates advanced work rather than replacing foundational learning.

Our current approach includes several safeguards: (1) **In-class code presentation and defense**, where students must explain their design decisions, justify alternative selections, and demonstrate understanding of signal flow and parameter relationships. (2) **Design tree documentation**, requiring students to articulate why they chose specific synthesis approaches over alternatives, revealing conceptual understanding. (3) **Live modification challenges**, where students must debug, modify, or extend their code in real-time without Dr.C assistance, testing foundational knowledge. (4) **From-scratch assignments**, periodically requiring manual Csound coding to ensure students maintain core technical skills.

The curriculum must balance acceleration with comprehension, ensuring Dr.C scaffolds learning rather than replaces it. Students must demonstrate they can "play the instrument," not merely operate the tool. This requires ongoing pedagogical research, assessment refinement, and honest acknowledgment that AI-assisted education demands new evaluation frameworks that

distinguish between tool-assisted creation and foundational mastery.

The Paradox of Accessibility: An ongoing case study with a graduate student—trained in other programming languages and audio tools but new to Csound—reveals a critical question: Does Dr.C motivate users to *learn* Csound deeply, or merely to *use* Csound productively? There exists a possibility that Dr.C could significantly expand Csound adoption while simultaneously reducing the proportion of users who develop deep understanding of synthesis, signal processing, and computer music design principles. Users may create more, accomplish more ambitious projects, yet learn less about underlying fundamentals. This is not necessarily a failure, but rather a fundamental shift in the relationship between tool mastery and creative output that must be recognized, studied longitudinally, and monitored over the coming years. The question remains open: Is a large community of AI-assisted Csound practitioners preferable to a smaller community of synthesis experts? The answer likely depends on educational context, creative goals, and the evolving role of AI in creative practice.

7. REAL-WORLD DEPLOYMENTS

Six applications built with Dr.C, deployed at <https://github.com/csounder>:

1. **Étude 1:** Generative ambient composition [29] with 14 autonomous instrumental layers and audio-reactive 3D particle visualization
2. **Weather Sonification:** Maps real-time weather API data to Csound parameters
3. **Fractal Music Explorer:** L-system grammar [26] to Csound score generation
4. **Mandelbrot Explorer:** Mandelbrot set [25] sonification with zoom navigation
5. **Memory Maker:** Multimedia slideshow creator for music therapy combining photos, videos, and music. Version 2 will add sing-along features with pitch correction via Csound’s phase vocoder opcodes
6. **Csound Drum Machine:** Pattern and arrangement sequencer allowing users to string together patterns and sub-patterns into complete songs. Features customizable drum kits with individual timbre, envelope, and balance controls per sound, plus effects including resonators, ring modulators, and multi-tap delays

Dr.C facilitated the creation, customization, and modification of these applications in hours and days rather than weeks and months. Applications are being deployed in Berklee courses and demonstrate Dr.C’s ability to scaffold complex generative systems, algorithmic composition, therapeutic applications, and rhythm programming. Figures 4–9 show screenshots of selected deployed applications.

8. ETHICAL CONSIDERATIONS

8.1. Author-as-Curator Model

Dr.C’s knowledge base is curated from the authors’ published works and expert-selected community resources including *The Csound Book* [1], foundational texts on spectral processing [3], computer music [4, 5], electronic music history [6], ambient composition [29, 11, 12], composition techniques [7, 8, 9], and musical form [10], plus the Classic Boulanger Csound Instrument Catalog, Amsterdam Catalog [17], FLOSS Manual [15] and examples [16], Csound Journal [21], Csound Magazine [22], Cabbage Manual [19] and forum [20], CsoundQt manual and examples [24], and the Iain McCurdy Instrument Collection [23] and Gogins collection [18]. This “author-as-curator” model ensures:



Figure 4: *Étude 1: Generative ambient composition with 14 autonomous instrumental layers (left panel) and audio-reactive 3D particle visualization responding to live audio levels.*

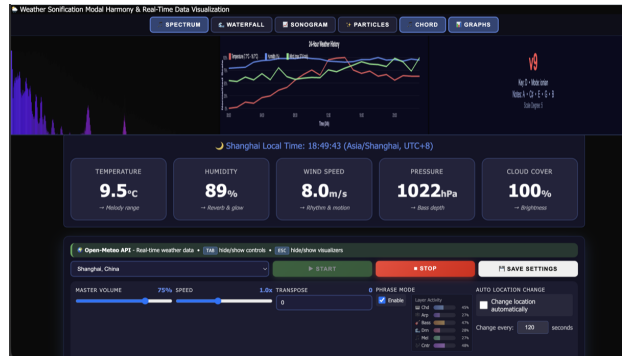


Figure 5: *Weather Sonification: Real-time weather API data mapped to Csound synthesis parameters including temperature, pressure, wind speed, and humidity.*

- Explicit consent (author controls training data)
- Attribution preserved in retrieval
- Educational intent primary
- No corporate web scraping

8.2. Open Source Commitment

Dr.C is MIT-licensed, works with local models (Ollama), and designed for FLOSS ecosystem integration. All deployed applications are open-source with full CSD code visible.

8.3. Human Agency

Alternatives-first design ensures humans remain in creative control. Dr.C is a *collaborator*, not autonomous generator. Design trees document human decision-making, not just AI output.

9. RELATED WORK

AI-Assisted Programming: GitHub Copilot, Cursor, and Replit Agent provide general-purpose code completion but lack domain specialization, semantic validation, and alternatives-first interaction.

Music AI Systems: Magenta, AIVA, and MuseNet generate musical content but do not scaffold *learning* or provide *tool-level* assistance for synthesis programming.

Csound Education: CsoundQt [24], Web-Csound, and Cabbage provide IDEs but no intelligent assistance. Blue’s pattern language offers abstraction but not AI collaboration.

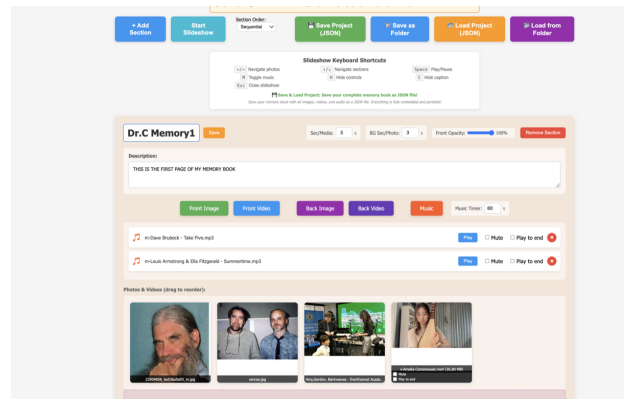
Dr.C uniquely combines domain-specialized AI with educational scaffolding and FLOSS principles.

10. FUTURE WORK

We are also integrating the Csound Reference Manual and FLOSS Manual into Dr.C’s knowledge retrieval for real-time opcode documentation and working examples during the design process.

Dr.C demonstrates that domain-specialized AI assistants can significantly outperform general-purpose tools for creative technical work in FLOSS audio. The alternatives-first paradigm, multi-tier knowledge retrieval, semantic validation, and design tree exploration represent transferable patterns for other specialized domains.

Dr.C is freely available under MIT license, representing a community resource for Csound education and a reference implementation for domain-specialized AI in FLOSS audio development.



The screenshot displays the DrC - Csound Drum Machine software interface. At the top, there's a header with 'Pattern View', the title 'DrC - Csound Drum Machine', and 'Arrangement View'. Below the header is a control bar featuring buttons for 'Load', 'Save', 'Export', 'Print', and 'Help', along with a '60%' zoom level. The main workspace is divided into three primary sections. The top section, 'Global Effects', contains seven sliders for parameters such as 'Reverb', 'Delay', 'Distortion', 'Tilt', 'Filter', 'LFO', and 'Tilt'. The middle section is the 'Pattern Editor', which consists of a grid of 16 columns and 16 rows for triggering notes. The bottom section, 'Instrument Controls', includes sliders for 'Attack', 'Decay', 'Release', and 'Sustain', along with a 'Volume' knob. The interface is dark-themed and uses a color-coded system for notes and effects.

12. ACKNOWLEDGMENTS

13. REFERENCES

- [1] R. Boulanger, Ed., *The Csound Book: Perspectives in Software Synthesis, Sound Design, Signal Processing, and Programming*. MIT Press, 2000.
- [2] V. Lazzarini, S. Yi, J. Heintz, Ø. Brandtsegg, and I. McCurdy, *Csound: A Sound and Music Computing System*. Springer, 2016.
- [3] V. Lazzarini, *Spectral Music Design: A Computational Approach*. Oxford University Press, 2017.
- [4] C. Roads, *The Computer Music Tutorial*. MIT Press, 1996.
- [5] C. Roads, *Composing Electronic Music: A New Aesthetic*. Oxford University Press, 2015.
- [6] J. Chadabe, *Electric Sound: The Past and Promise of Electronic Music*. Prentice Hall, 1997.
- [7] D. Cope, *Techniques of the Contemporary Composer*. Schirmer Books, 1997.
- [8] D. Cope, *New Directions in Music*. Waveland Press, 2001.
- [9] D. Cope, *Virtual Music: Computer Synthesis of Musical Style*. MIT Press, 2004.

- [10] H. Norden, *Form: The Silent Language*. Crescendo Publishing, 1969.
- [11] D. Warner, *Composing Ambient Music: Inspirational Strategies for Creating Ambient Electronic Music*. Self-published, 2017.
- [12] M. Pendergrast, *The Ambient Century: From Mahler to Trance—The Evolution of Sound in the Electronic Age*. Bloomsbury, 2003.
- [13] "Csound — A Sound and Music Computing System," <https://csound.com/>, accessed March 2026.
- [14] "The Csound Reference Manual," <https://csound.com/docs/manual/index.html>, accessed March 2026.
- [15] J. Heintz, A. Hofmann, and I. McCurdy, "FLOSS Manual — Csound," <https://flossmanual.csound.com/>, accessed March 2026.
- [16] "FLOSS Manual Csound Examples," <https://github.com/csudoc/csudoc.github.io>, accessed March 2026.
- [17] "The Amsterdam Catalog of Csound Computer Instruments," <http://www.codemist.co.uk/AmsterdamCatalog/>, accessed March 2026.
- [18] M. Gogins, "Extended Csound and Web Audio Examples," <https://michaelgogins.tumblr.com/>, accessed March 2026.
- [19] R. Walsh, "Cabbage Audio — Documentation and Examples," <https://cabbageaudio.com/docs/>, accessed March 2026.
- [20] "Cabbage Audio Forum," <https://forum.cabbageaudio.com/>, accessed March 2026.
- [21] "The Csound Journal," <https://csoundjournal.com/>, accessed March 2026.
- [22] "The Csound Magazine," <http://www.csounds.com/ezine/>, accessed March 2026.
- [23] I. McCurdy, "Iain McCurdy Csound Instrument Collection," <http://iainmccurdy.org/csound.html>, accessed March 2026.
- [24] A. Cabrera, "CsoundQt Examples," <https://github.com/CsoundQt/CsoundQt/tree/develop/src/Examples>, accessed March 2026.
- [25] B. Mandelbrot, *The Fractal Geometry of Nature*. W. H. Freeman, 1983.
- [26] P. Prusinkiewicz and A. Lindenmayer, *The Algorithmic Beauty of Plants*. Springer-Verlag, 1990.
- [27] E. Varèse, *Poème électronique*. Philips Pavilion, Brussels World's Fair, 1958.
- [28] J. Chowning, *Stria*. Stanford University, 1977.
- [29] B. Eno, *Ambient 1: Music for Airports*. Polydor Records, 1978.
- [30] I. McCurdy, *Csound Haiku*. Self-published collection, 2012.
- [31] BT (Brian Transeau), *This Binary Universe*. Nettwerk Records, 2005.